Phase 3: Project Design Phase

This phase establishes the structural blueprint of the **shopEZ** platform, defining the architectural pattern, data flows, database schemas, and problem-solution maps.

---

1. Problem - Solution Fit Matrix

To address the target audience's core complaints, shopEZ introduces modern MERN stack patterns:

Customer Pain Points	Existing Alternatives (e.g. Shopify, Monoliths)	shopEZ Solution Fit
Cart desync & item loss	Session-only cookies or basic local storage which clears on session timeout.	**Persistent MERN Lifecycle:** State is bound to React Context, syncing with local storage and database endpoints on login.
Poor catalog search/filtering	Simple string matches that ignore variants and return heavy, unfiltered datasets.	**Dynamic Backend Filters:** Mongoose queries support autocomplete suggestions, regex search, pricing boundaries, and rating filters.
Unoptimized media loading	Local server hosting which bottlenecks bandwidth and loads high-resolution files.	**Cloudinary CDN Pipeline:** Uploaded images are offloaded to Cloudinary, serving optimized images based on user viewport.
Complex administrative controls	Split admin pages or generic CMS dashboards with no real-time inventory alerts.	**Unified Admin Ledger:** Live analytics displays monthly earnings graphs, user toggles, and low-stock indicators.

---

2. Proposed Solution Details

The shopEZ marketplace represents a modern full-stack implementation designed to balance consumer satisfaction with business administrative control.

- **Novelty & Uniqueness:** By decoupling the React client-side application from the Node.js API server, we enable instant page switching, localized micro-animations, and background computations. The admin panel compiles inventory and user access in real-time, removing the heavy reload times typical of monolithic e-commerce CMS installations.

- **Customer Satisfaction:** A transparent, itemized billing summary at checkout calculations (base price, coupon discount, shipping method cost, tax percentages) builds high customer trust. Real-time review checks and helpful voter tallies allow buyers to make reliable purchasing choices.
- **Business Model:** Monetized via direct product margins and seller product slots. Standard administrative controls facilitate low-stock alerts to minimize sales loss.

---

3. Solution Architecture (MVC Design Pattern)

The system enforces strict decoupling of client-server application flows using the Model-View-Controller (MVC) design pattern mapped across the codebase:

```
graph TD
    Client["Client Browser (React SPA View)"]
    Server["Express Web Server (Node.js Controller)"]
    Auth["JWT/Auth Middleware"]
    DB[("MongoDB Atlas (Mongoose Model)")]
    Cloudinary["Cloudinary API (Asset CDN)"]
    Stripe["Mock Stripe Payments"]

    Client -->|1. REST API Requests (Axios)| Server
    Server -->|2. Route Guarding| Auth
    Auth -->|3. Query Validation| Server
    Server -->|4. Read/Write Models| DB
    Server -->|5. Upload Media| Cloudinary
    Client -->|6. Payment Intents| Stripe
```

1. Frontend (View)

- **Technology Stack:** React.js + Vite + Tailwind CSS.
- **Role:** Serves the dynamic Single Page Application (SPA) to the browser.
- **Key Components:**
- **Context State Providers:** Maintain user session state (AuthContext), toast notifications, and persistent shopping cart items.
- **Product Views:** Render grid search lists with sliders, related product lists, and review feeds.
- **Interactive Wizard Checkout:** Handles shipping location forms, coupon validation flags, and Stripe inputs.

2. Backend (Controller)

- **Technology Stack:** Node.js + Express.js.
- **Role:** Processes incoming RESTful API routes, validates payload bodies, and serves structured JSON responses.

- **Key Controllers:**
- authController.js: Coordinates registrations, hashes passwords via Bcrypt, and compiles addresses/wishlists.
- productController.js: Formulates catalog search queries, aggregates stars, and manages review logs.
- orderController.js: Coordinates stock adjustments, processes payments, and updates fulfillment timelines.
- couponController.js: Evaluates coupon expiration dates, active toggles, and discounts.

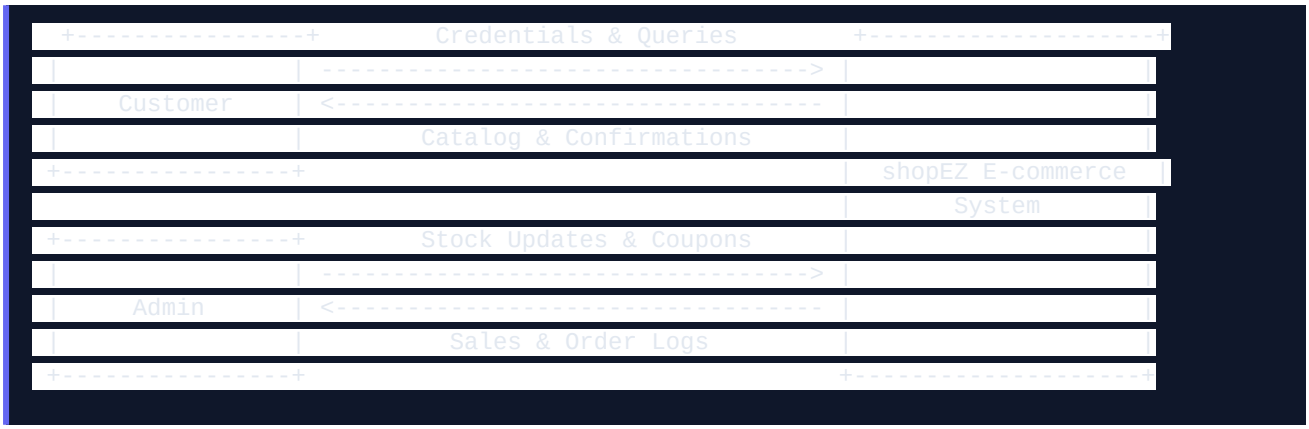
3. Database (Model)

- **Technology Stack:** MongoDB + Mongoose ODM.
- **Role:** Houses the collections representing the application state.
- **Models Defined:**
- User.js: User profiles (name, email, role, wishlist array, saved shipping addresses array).
- Product.js: Inventory listings (title, description, price, stock quantity, brand, category, rating score, reviews list).
- Order.js: Financial and tracking items (purchased items, shipping location, shipping rate, tax, total cost, payment status, status tracking timeline).
- Coupon.js: Discount entries (code, percentage/fixed value, status, expiry dates).

---

4. Data Flow Diagrams (DFD)

Level 0 DFD: Context Diagram



Level 1 DFD: Process Breakdown

1. **User Authentication Flow:**
2. Customer inputs email/password -> Express Router -> authMiddleware.js -> Database User validation -> Server returns JWT token stored in cookie.

3. **Product Browsing Flow:**
4. React catalog triggers `GET /api/products?search=...` -> productController.js -> Database text search with filters -> Server returns filtered product list.
5. **Cart Checkout Flow:**
6. Customer places order -> React initiates `POST /api/payment/create-intent` -> Server processes mock Stripe transaction -> Server saves Order to Order.js -> Server decrements Product stock counts in Product.js.